Tema 9:

Grafos

GRADO EN INGENIERÍA DEL SOFTWARE

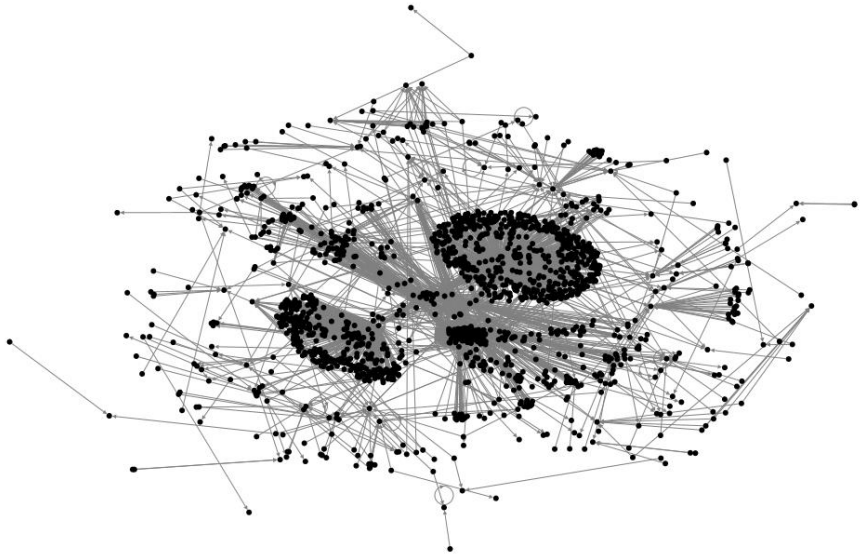
Salvador Sánchez y Cristian Rodríguez

Universidad
Rey Juan Carlos

Curso 2023-2024

Contenidos

- Introducción
  - Definición de grafo
  - Tipos de grafos
  - Conceptos básicos
- Representación
- Implementación
- Recorridos
- Ejercicios propuestos



Definición de grafo

- Un **grafo** es una estructura de datos abstracta, no lineal, compuesta por un conjunto de vértices o nodos y un conjunto de aristas o arcos que los conectan.

- Formalmente se define como un par G = (V,A), donde V es el conjunto de vértices y A es el conjunto de arcos.
  - Un arco es un par (v_1,v_2) de vértices de V.

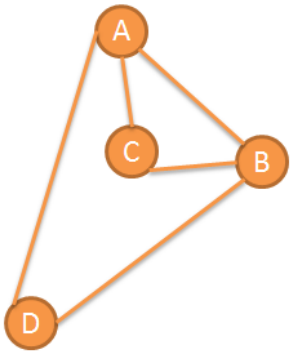
- Los grafos se utilizan para modelar numerosos dominios tales como mapas, transporte o redes de todo tipo (de ordenadores, de comunicación, sociales, biológicas, etc.).

Tipos de grafos

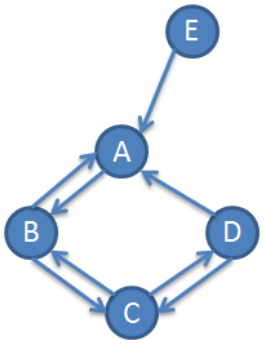
- Existen múltiples formas de clasificar los grafos, atendiendo a distintas características:
  - Dirigidos vs. No dirigidos
  - Binarios vs. Ponderados (valorados)
  - Conexos vs. No conexos
  - Unicapa vs. Multicapa (múltiplex)
  - Simples vs. multigrafos
  - Unipartitos vs. Bipartitos
  - Homogéneos vs. Heterogéneos
  - Estáticos vs. Dinámicos(temporales)
  - Etc.

Grafos dirigidos / no dirigidos

- En un grafo dirigido (o “digrafo”) las aristas son pares ordenados con un sentido definido (`origen,destino`).
- Cuando las aristas no son pares ordenados, estamos ante un grafo no dirigido.



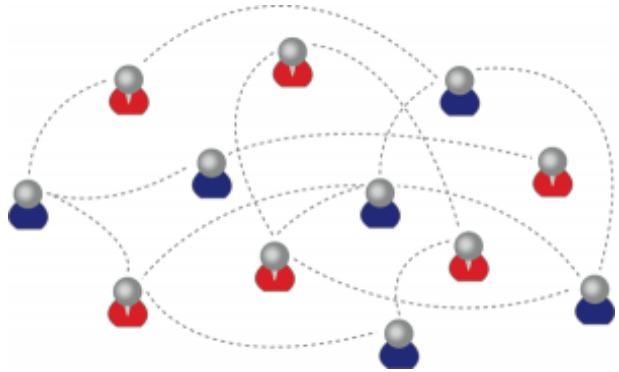
no dirigido



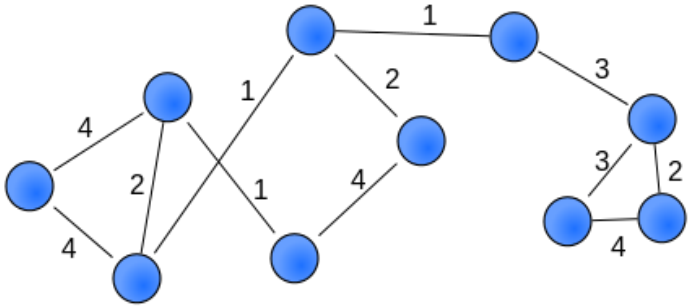
dirigido

Grafos binarios / valorados

- Un grafo se denomina binario cuando las aristas entre nodos existen o no existen, pero no está cuantificada la calidad o valor de la relación.
- En los grafos ponderados la relación entre dos nodos se puede cuantificar con un peso determinado



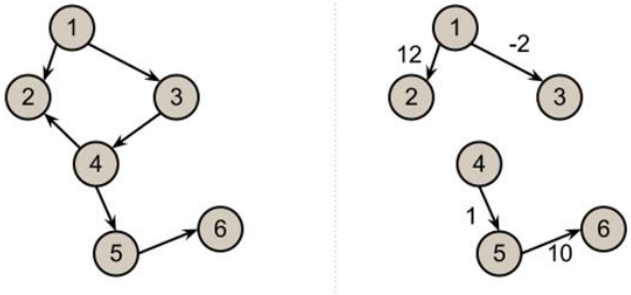
Grafo binario



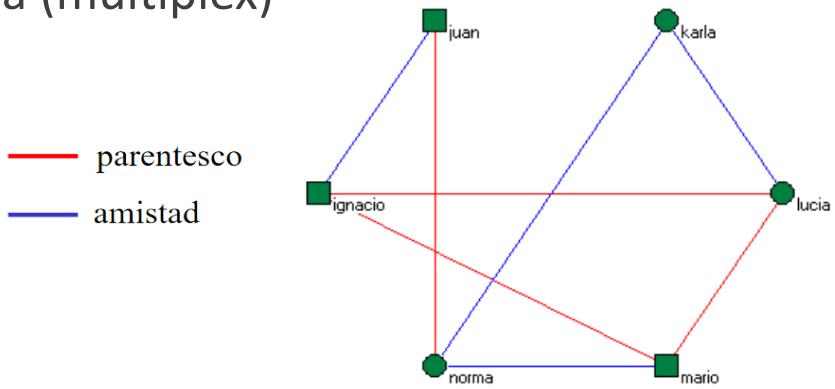
Grafo ponderado

Otras clasificaciones

- Conexo vs. no conexo

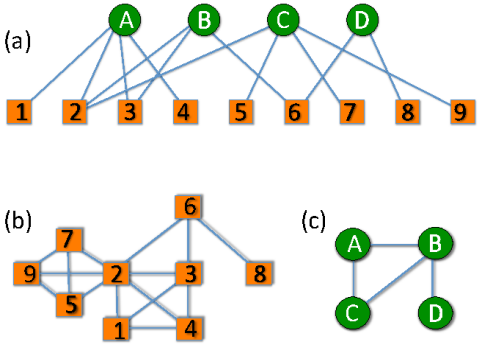


- Unicapa vs. Multicapa (múltiplex)

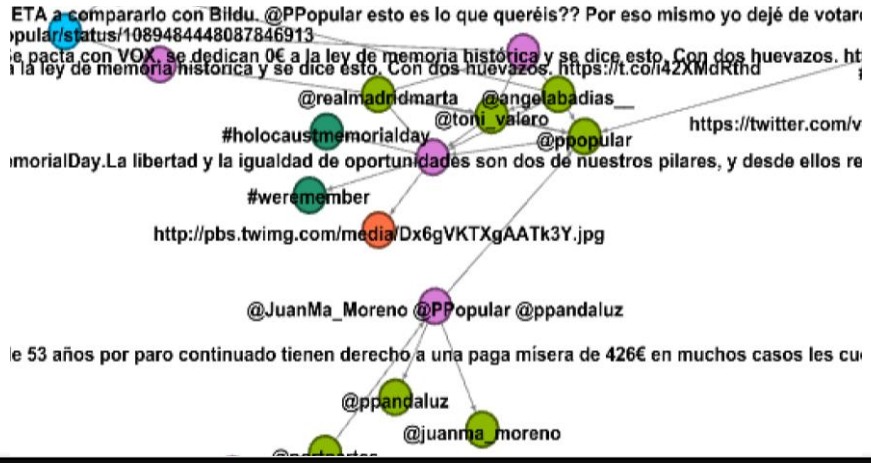


Otras clasificaciones

- Unimodales vs. Bimodales

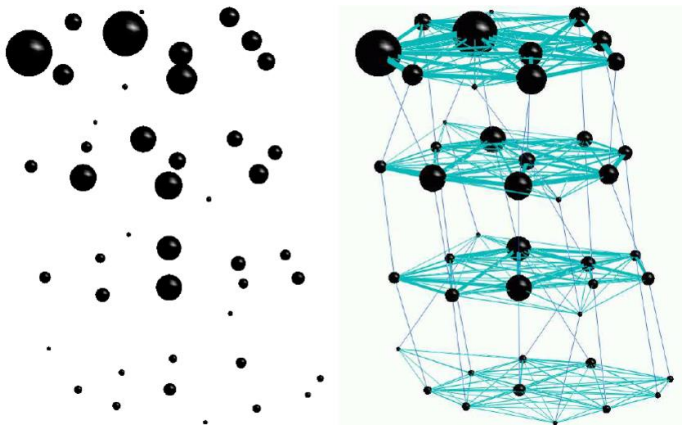


- Homogéneos vs. heterogéneos



Otras clasificaciones

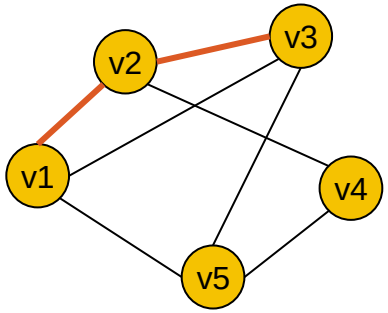
- Grafos dinámicos/temporales:



- Grafo simple vs. multigrafo: si se permiten una o más aristas entre nodos

Conceptos básicos

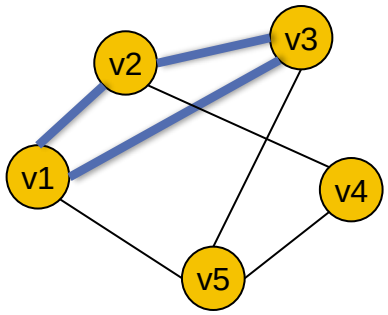
- **Grado** de un vértice: número de aristas
  - Grado saliente(**out-degree**)/entrante (**in-degree**) en grafos dirigidos
- **Camino**: Secuencia de vértices entre un origen y un destino
  - **Camino simple**: todos sus nodos son distintos
  - **Longitud**: número de aristas de un camino
  - **Camino cerrado**: el primer y último bucle son el mismo
  - **Camino mínimo**: el más corto entre dos vértices



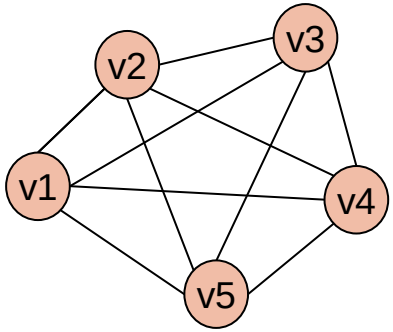
Caminos de v1 a v3:
- v1-v3 (simple, mínimo, longitud 1)
- v1-v2-v3 (simple, longitud 2)
- v1-v5-v4-v2-v3 (simple, longitud 4)
- v1-v2-v4-v5-v1-v2-v3 (longitud 6)

Conceptos básicos

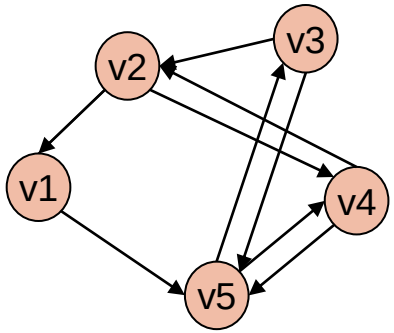
- **Adyacencia**: se dice de vértices unidos por un arco
- **Bucle**: camino simple cerrado que puede tener longitud k
- **Grafo completo**: incluye todas las posibles aristas entre sus vértices
- **Grafo fuertemente conexo**: existe un camino desde cualquier nodo a cualquier otro



Bucle de longitud 3



Grafo completo



Grafo fuertemente conexo

TAD Grafo: especificación

- **Nombre:** Grafo dirigido / valorado / etc.
- **Valores posibles:** *vacío, otros (según elementos)*.
- **Operaciones:**
  - `insert_node()`
  - `insert_edge()`
  - `delete_node()`
  - `delete_edge()`
  - `is_empty(graph)`
  - `in_graph(node) / in_graph(arc)`
  - `degree/indegree/outdegree(node)`
  - `adjacent(node,node)`
- **Uso de otros tipos:** tipo base, punteros, arrays

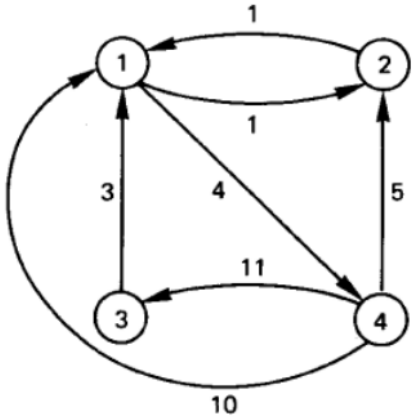
TAD Grafo: otras operaciones

- Un grafo es una estructura tan compleja y con usos tan distintos que existe un sin número de operaciones aplicables
- En análisis de redes sociales, por ejemplo, es común calcular métricas como:
  - `diameter`
  - `clustering coeficient`
  - `neighbourhood`
  - `betweenness, closeness, eigenvector centrality`
  - `density`
  - etc.
- Y operaciones como `extract_subgraph, distance(n1,n2), k-core, components, etc.`

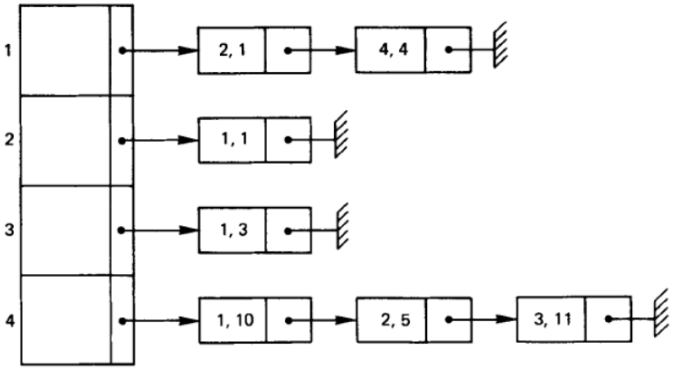
Representación

Representación

- Los grafos son estructuras complejas para cuya representación se emplean otras estructuras de datos
  - Representaciones **estáticas**: Matriz de adyacencia + vector nodos
  - Representaciones **dinámicas**: Lista de adyacencia, lista de arcos

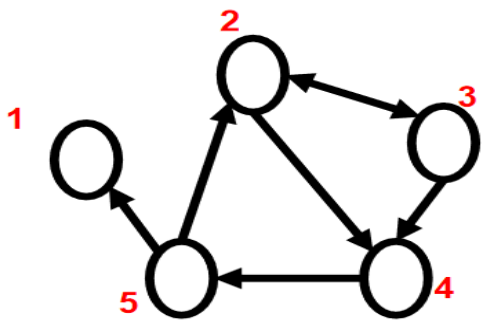


	1	2	3	4
1	∞	1	∞	4
2	1	∞	∞	∞
3	3	∞	∞	∞
4	10	5	11	∞



Representación estática

- **Matriz de adyacencia**: representación tabular con tantas filas y columnas como nodos tiene la red
  - Las celdas contienen información sobre la relación (los valores en la diagonal representan autoenlaces).
  - En grafos ponderados las celdas tienen valores numéricos (matriz etiquetada)
  - En grafos no dirigidos habrá una relación en ambos sentidos

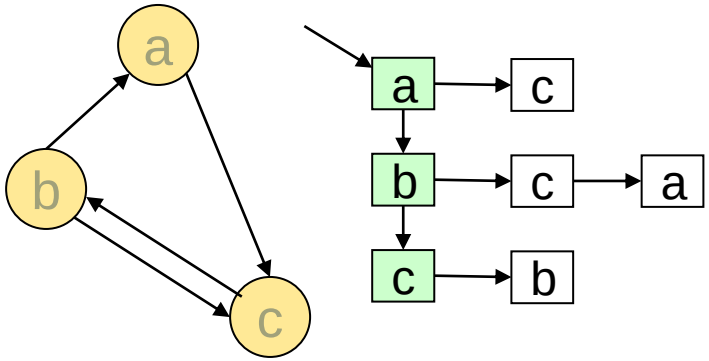


$$A = \begin{matrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 \end{matrix}$$

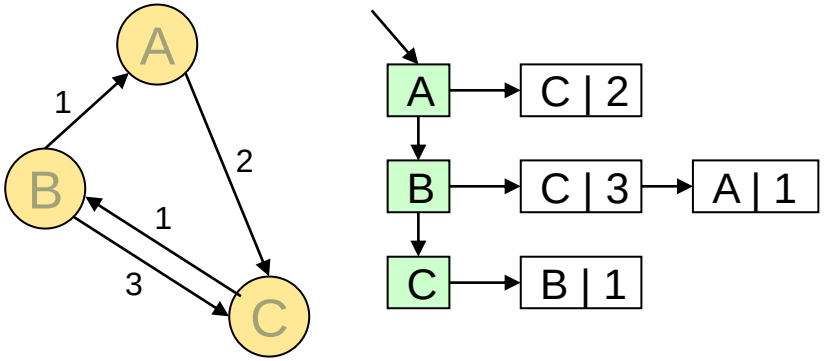
V = (1,2,3,4,5)

Representación dinámica

- **Lista de adyacencia**: Representación en forma de lista de todos los vecinos de cada nodo.
  - De cada nodo, sale una lista de adyacentes que puede estar ponderada o no
  - Incluye una fila por nodo y en dicha fila se listan los otros nodos con los que éste tiene una relación.



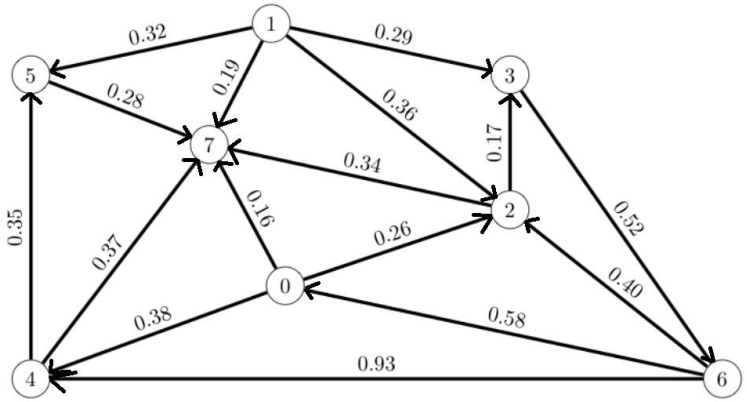
Grafo dirigido binario

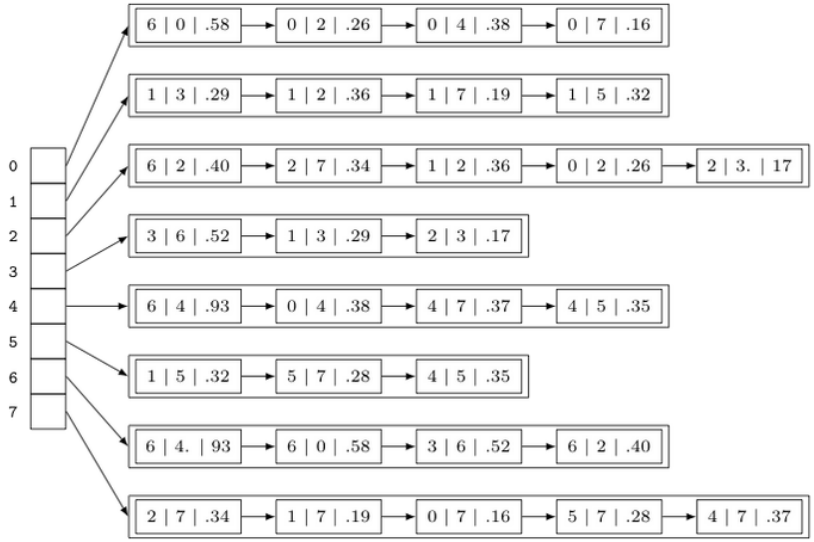


Grafo dirigido valorado

Representación dinámica

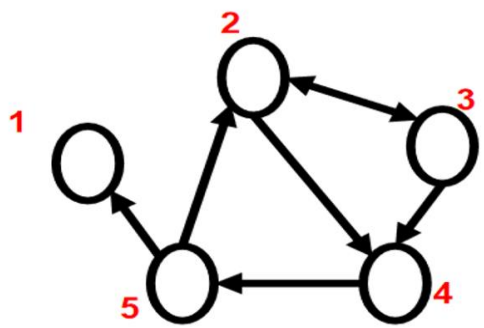
- Aunque la implementación anterior es la más habitual, existen variantes de la implementación de lista de adyacencia
- P.ej. una **lista de adyacencia con arcos asociados** donde en lugar de asociar a cada nodo los vértices que salen de él se le asocian todos los que salen o entran en él
  - Se asumen las redundancias
  - Útil según la aplicación





Representación dinámica

- **Lista de arcos**: Representación simplificada que incluye información explícita de todas las parejas origen-destino.
  - Puede ampliarse para representar redes valoradas añadiendo un tercer componente a cada pareja: tripletas origen, destino, valor.



- 2, 3
- 2, 4
- 3, 2
- 3, 4
- 4, 5
- 5, 2
- 5, 1

Comparación de representaciones

- Para un grafo G(V,A) las operaciones básicas se pueden comparar en función de las diferentes implementaciones:

Representación	espacio	añadir arco / nodo	¿arco entre n y m?	Iterar el vecindario de un nodo
Lista de arcos	**A**	**1**	**A**	**A**
Matriz de adyacencia	**V^2**	**1**	**1**	**V**
Lista de adyacencia	**V+N**	**1**	**grado(n)**	**grado(n)**

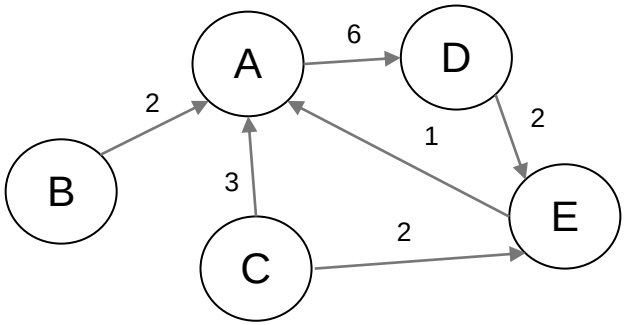
Comparación de representaciones

- En grafos poco densos (muchos nodos con pocas conexiones) la lista de adyacencia se muestra como la mejor opción.
  - La mayoría de los grafos extraídos de relaciones sociales tienen esta característica
  - Facilitan la determinación y búsqueda de caminos
- Para representar grafos densos con pocos nodos la opción más habitual es matrices de adyacencia.
  - Permite conocer los vecinos de un nodo de manera sencilla.

Implementación

Digrafo valorado en Matriz de adyacencia

```
unit uMatrixEdgeWeightedDigraph;

interface

  const
    MAX_NODES = 1000;

  type

    edgeWeightedDigraph = record
       matrix : array[1..MAX_NODES,1..MAX_NODES] of string;
       node_labels : array[1..MAX_NODES] of string;
       num_nodes: integer;
    end;
```



matrix

	1	2	3	4	5
1	∞	∞	∞	6	∞
2	2	∞	∞	∞	∞
3	3	∞	∞	∞	2
4	∞	∞	∞	∞	2
5	1	∞	∞	∞	∞

node_labels = (A,B,C,D,E)

num_nodes = 5

Digrafo valorado en matriz de adyacencia

```
unit uMatrixEdgeWeightedDigraph;

interface

  const
    MAX_NODES = 1000;

  type

    edgeWeightedDigraph = record
       matrix : array[1..MAX_NODES,1..MAX_NODES] of string;
       node_labels : array[1..MAX_NODES] of string;
       num_nodes: integer;
    end;

  procedure initialize(var g: edgeWeightedDigraph);
  procedure insert_node(var g: edgeWeightedDigraph; node:string);
  procedure insert_edge(var g: edgeWeightedDigraph; n1,n2,weight: string);
  procedure delete_node(var g: edgeWeightedDigraph; node:string);
  procedure delete_edge(var g: edgeWeightedDigraph; n1,n2:string);

  function is_empty(g: edgeWeightedDigraph): boolean;
  function node_in_graph(g: edgeWeightedDigraph; node: string): boolean;
  function edge_in_graph(g: edgeWeightedDigraph; n1,n2:string): boolean;
  function adjacent(g: edgeWeightedDigraph; n1,n2:string): boolean;
```

Digrafo valorado en matriz de adyacencia

```
implementation

  procedure initialize(var g: edgeWeightedDigraph);
  var i,j:integer;
  begin
    g.num_nodes:=0;
    for i:= 1 to MAX_NODES do begin
      g.node_labels[i]:= '';
      for j:= 1 to MAX_NODES do g.matrix[i,j]:= 'inf';
    end
  end;

  function is_empty(g: edgeWeightedDigraph): boolean;
  begin
    is_empty:= g.num_nodes=0
  end;
```

Digrafo valorado en matriz de adyacencia

```
function get_index(g: edgeWeightedDigraph; node:string): integer;
{ retorna el índice en el array de un nodo por su etiqueta }
var i:integer;
begin
  get_index:= -1;
  i:=1;
  while (get_index=-1) and (i<=MAX_NODES) do begin
    if g.node_labels[i] = node then get_index := i;
    i:= i+1;
  end
end;

procedure insert_node(var g: edgeWeightedDigraph; node:string);
begin
  if (g.num_nodes<MAX_NODES) and (not node_in_graph(g,node)) then begin
    g.num_nodes:= g.num_nodes + 1;
    g.node_labels[g.num_nodes]:= node;
  end;
end;

procedure insert_edge(var g: edgeWeightedDigraph; n1,n2,weight: string);
begin
  if node_in_graph(g,n1) and  node_in_graph(g,n2) then
      g.matrix[get_index(g,n1),get_index(g,n2)]:= weight;
end;
```

Digrafo valorado en matriz de adyacencia

```
function node_in_graph(g: edgeWeightedDigraph; node: string): boolean;
begin
  node_in_graph:= get_index(g,node)<>-1;
end;

function edge_in_graph(g: edgeWeightedDigraph; n1,n2:string): boolean;
var node1_index,node2_index: integer;
begin
  node1_index:= get_index(g,n1);
  node2_index:= get_index(g,n2);
  edge_in_graph:= (node1_index <> -1) and
                  (node2_index <> -1) and
                  (g.matrix[node1_index,node2_index]<>'inf');
end;
```

Digrafo valorado en matriz de adyacencia

```
function adjacent(g: edgeWeightedDigraph; n1,n2:string): boolean;
begin
  adjacent:= edge_in_graph(g,n1,n2);
end;
```

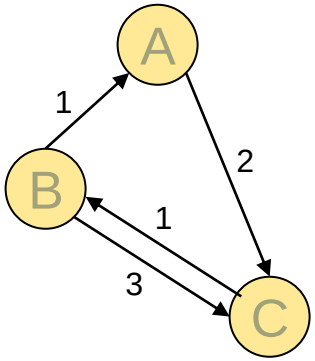
```
procedure delete_edge(var g: edgeWeightedDigraph; n1,n2:string);
begin
  if node_in_graph(g,n1) and node_in_graph(g,n2) then
    g.matrix[get_index(g,n1),get_index(g,n2)]:= 'inf';
end;
```

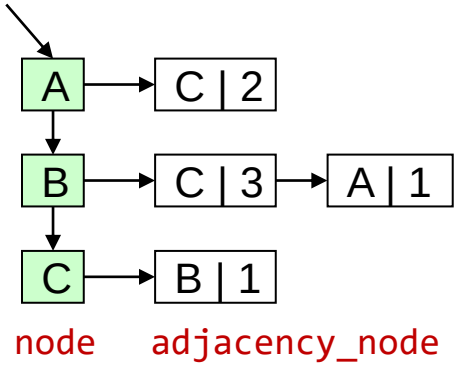
Digrafo valorado en matriz de adyacencia

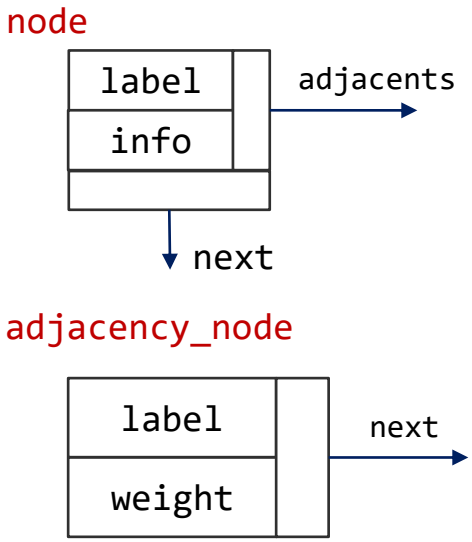
```
procedure delete_node(var g: edgeWeightedDigraph; node:string);
var i,k:integer;
begin
  if node_in_graph(g,node) then begin
    k:= get_index(g,node);
    if k=g.num_nodes then // caso especial: eliminar el último nodo
      for i:=1 to g.num_nodes-1 do begin
          g.matrix[i,g.num_nodes]:='inf';
          g.matrix[g.num_nodes,i]:='inf';
      end
    else begin // eliminar sobreescribiendo su información con la del último
      for i:=1 to g.num_nodes-1 do begin
          g.matrix[i,k]:=g.matrix[i,g.num_nodes];
          g.matrix[i,g.num_nodes]:='inf';
          g.matrix[k,i]:=g.matrix[g.num_nodes,i];
          g.matrix[g.num_nodes,i]:='inf';
      end;
      g.matrix[k,k]:=g.matrix[g.num_nodes,g.num_nodes];
      g.node_labels[k]:= g.node_labels[g.num_nodes];
    end;
    g.matrix[g.num_nodes,g.num_nodes]:='inf';
    g.num_nodes:=g.num_nodes-1;
  end;
end;
```

Digrafo valorado en lista de adyacencia

```
unit uAdjacencyListEdgeWeightedDigraph;

interface

  uses uTElemento;

  type

    tadjacencyNode = record
       node_label, weight: string;
       next: ^tadjacencyNode;
    end;

    tnode = record
       node_label: string;
       info: TElemento;
       adjacents: ^tadjacencyNode;
       next: ^tnode
    end;

    edgeWeightedDigraph = ^tnode;
```







Recorridos

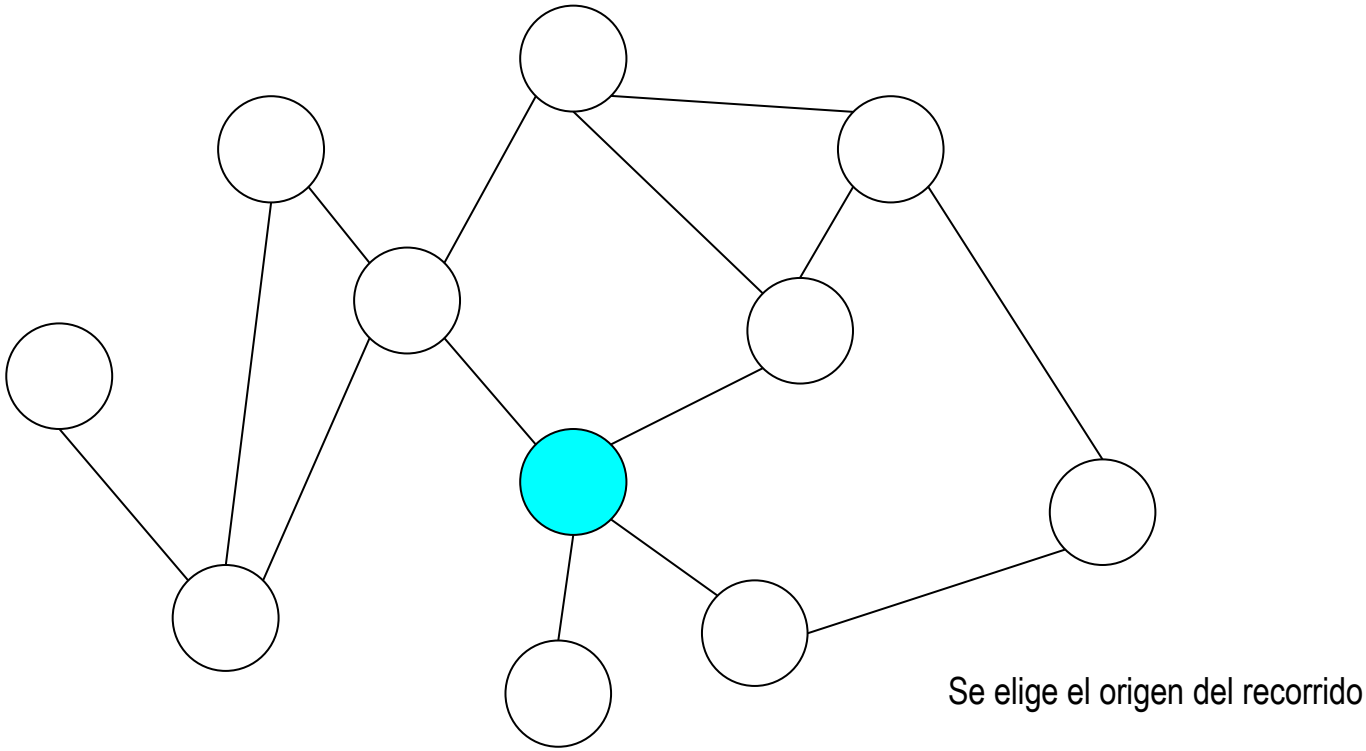
Recorridos

- Recorrer un grafo es visitar todos los nodos alcanzables a partir de uno dado sin repetir nodos.
- Pueden ser:
  - Recorridos en anchura o anchura (BFS, *Breadth-first*)
  - Recorrido en profundidad (DFS, *Depth-first*)
- En ambos tipos se necesita guardar información sobre los nodos que ya han sido visitados, para no volver a visitarlos

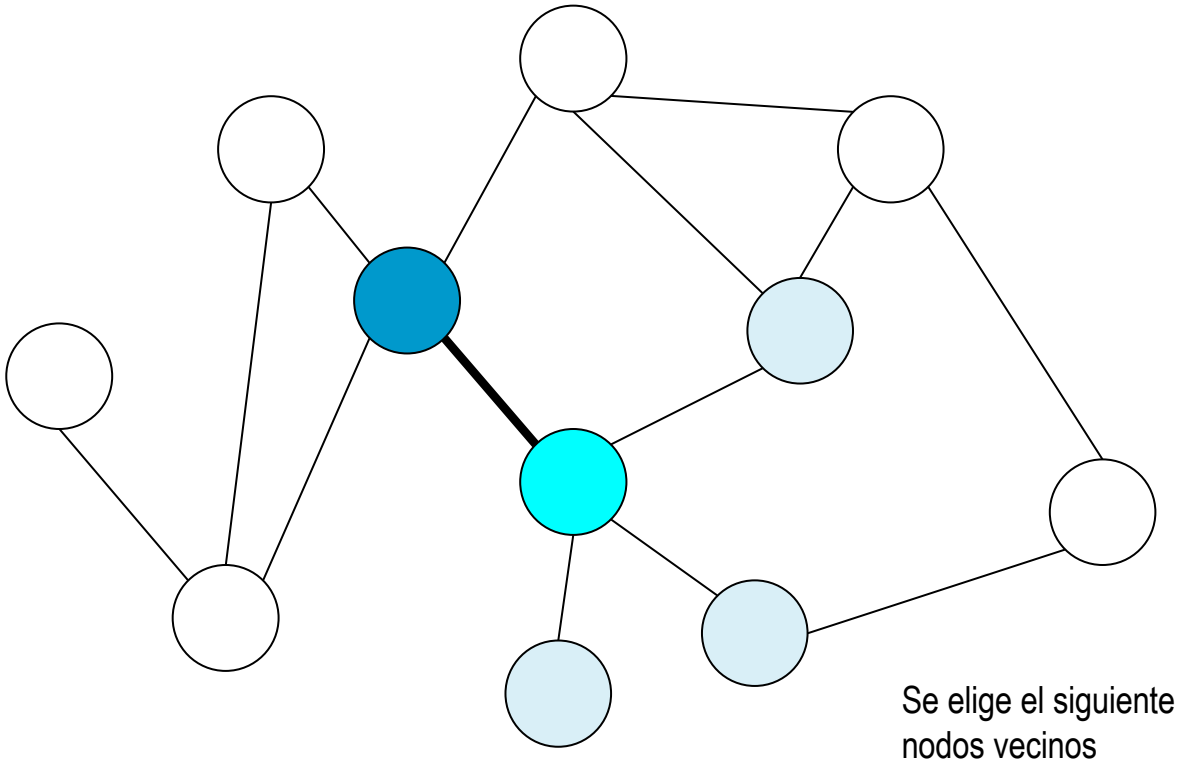
Recorrido en anchura (BFS)

- El orden de visita de los nodos del grafo es el siguiente:
  1. Visita primero los adyacentes del nodo actual
  2. Cuando ha terminado, procede con el resto comenzando por los adyacentes del primer nodo “pendiente”
- Para ello utiliza una cola auxiliar a la que añade en cada paso los adyacentes al nodo actual
- Marca los visitados para evitar bucles y repeticiones no deseadas

Recorrido en anchura

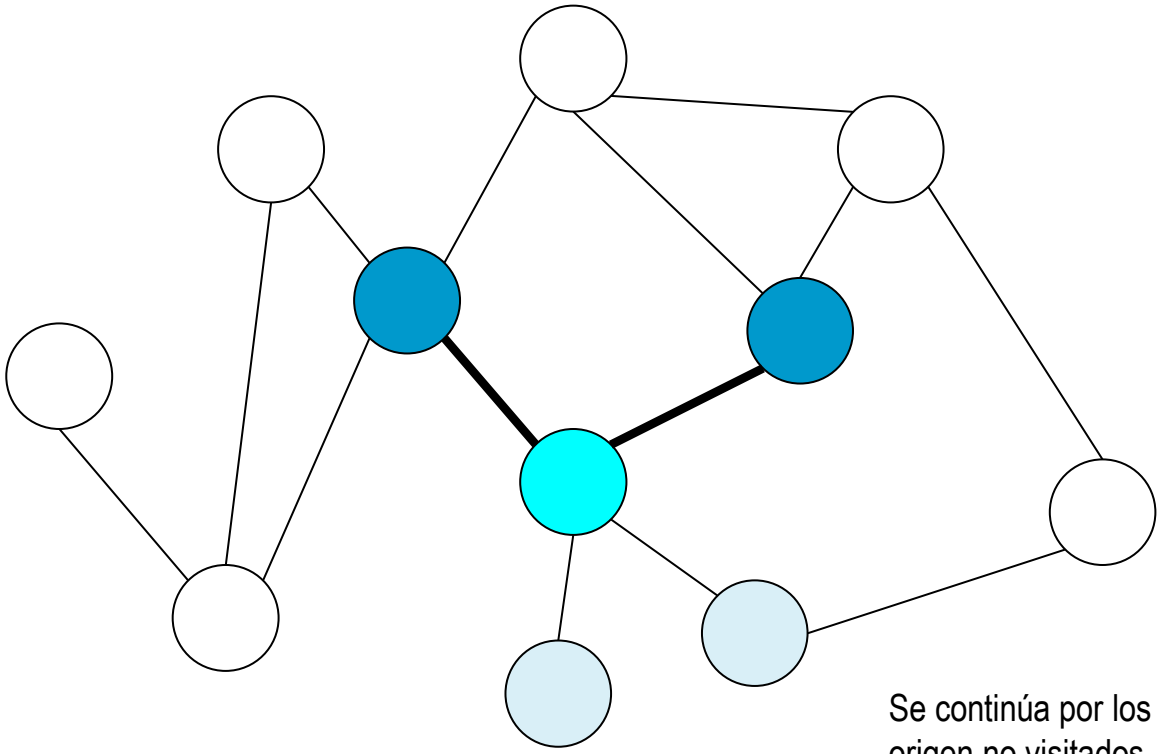


Recorrido en anchura



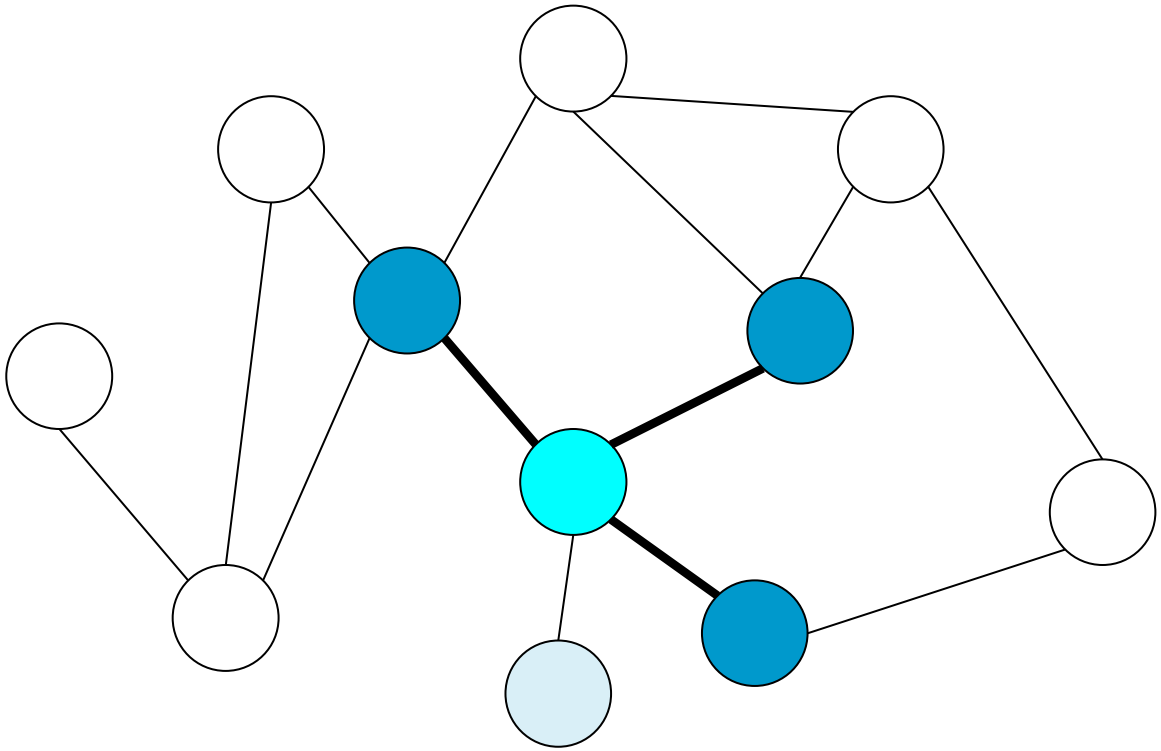
Se elige el siguiente a visitar entre sus nodos vecinos

Recorrido en anchura

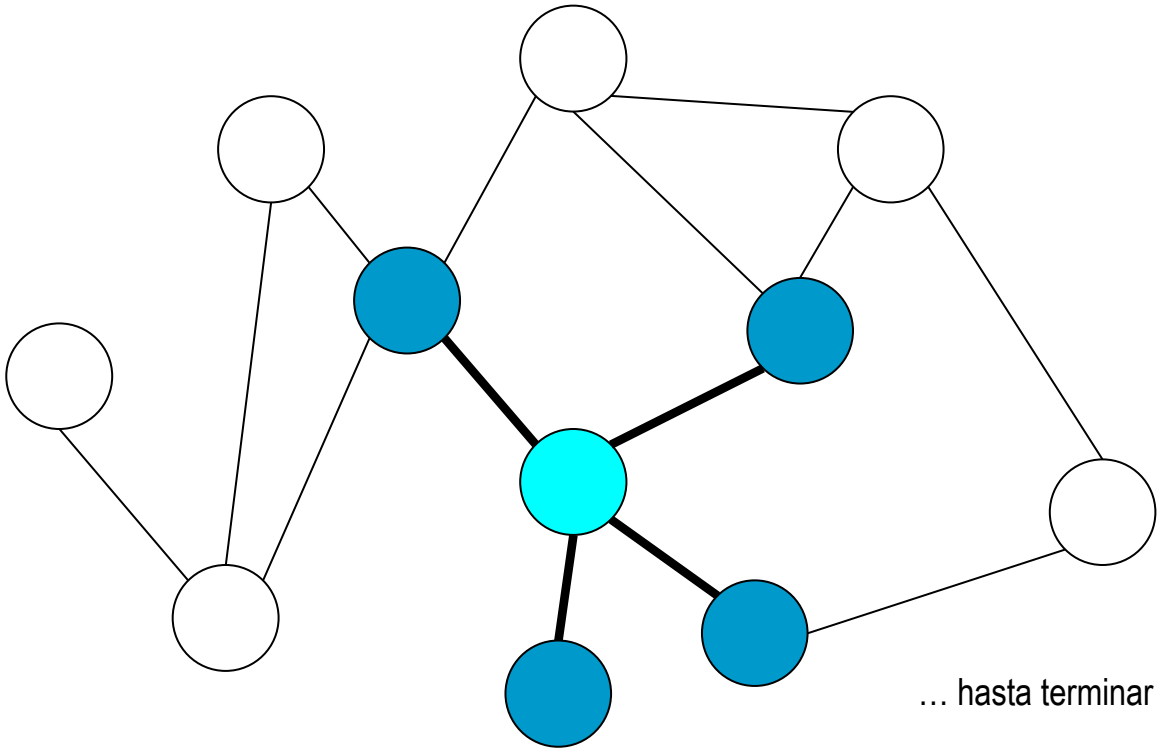


Se continúa por los restantes vecinos del origen no visitados…

Recorrido en anchura

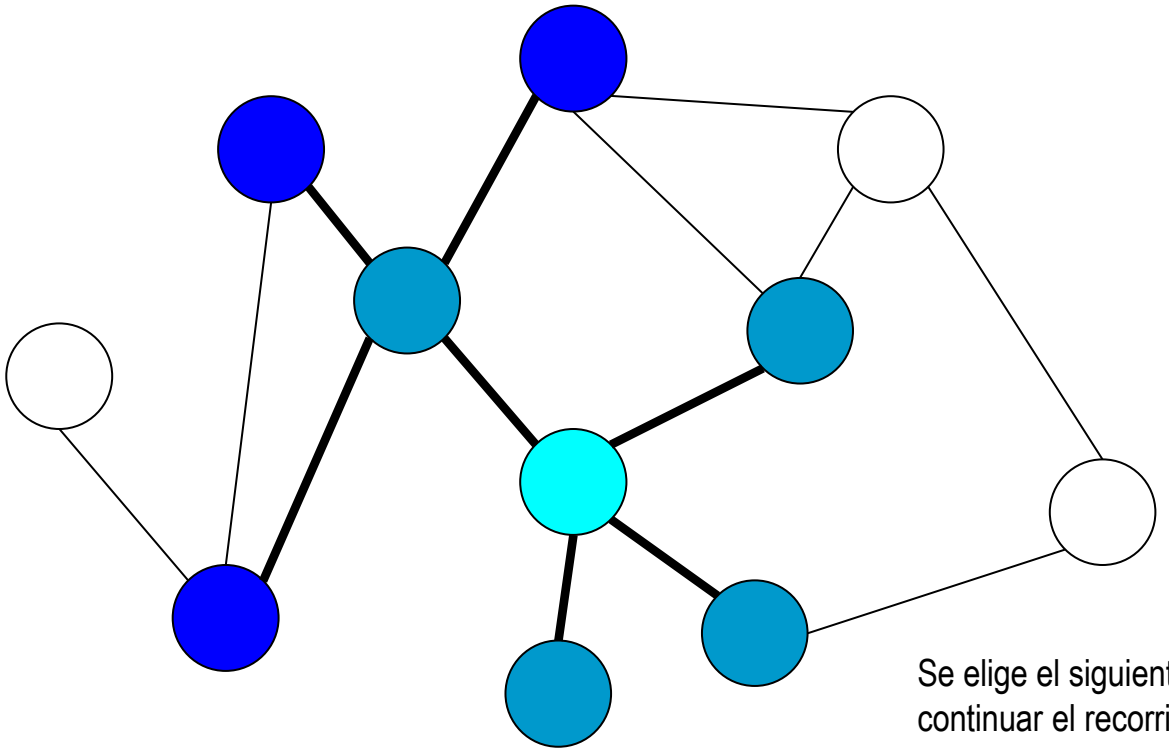


Recorrido en anchura



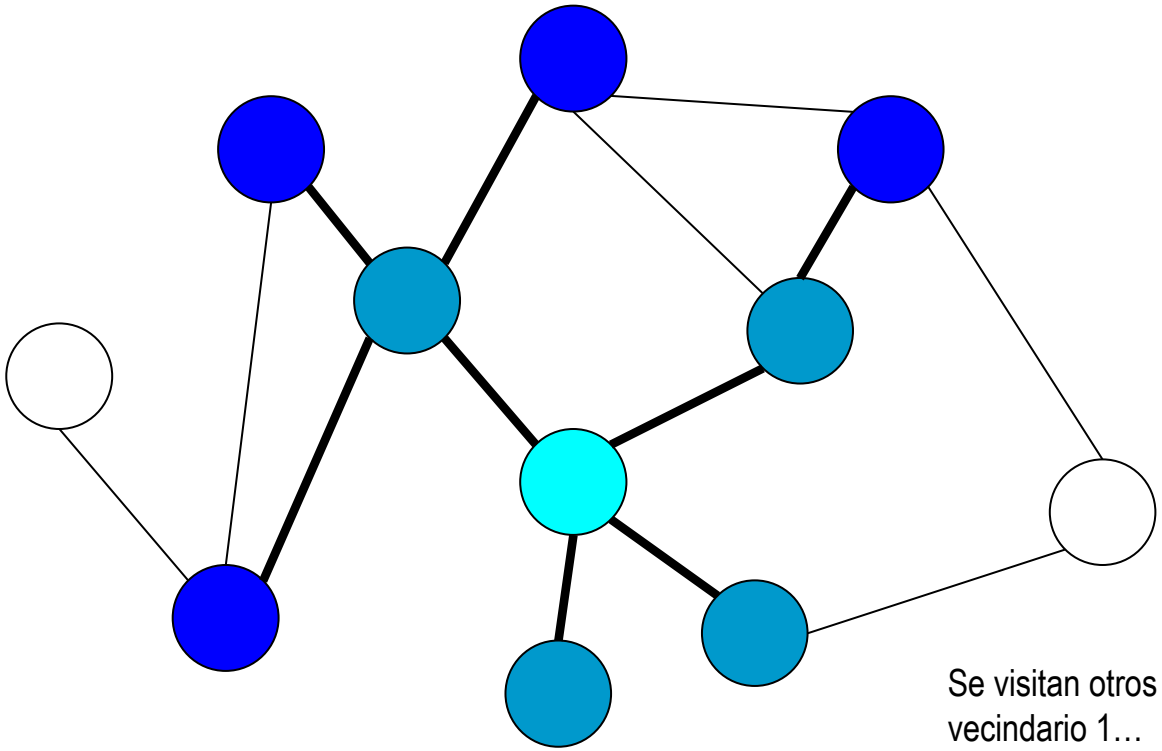
… hasta terminar con el vecindario de nivel 1

Recorrido en anchura



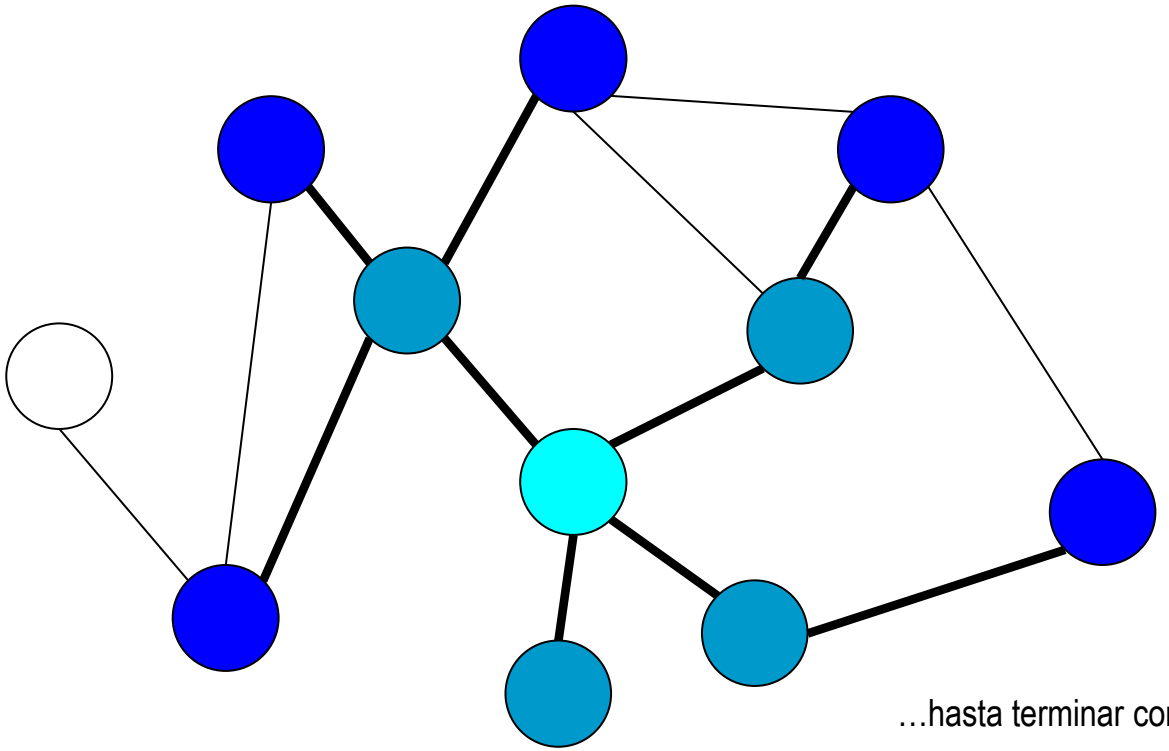
Se elige el siguiente nodo desde el que continuar el recorrido (vecindario nivel 2)

Recorrido en anchura



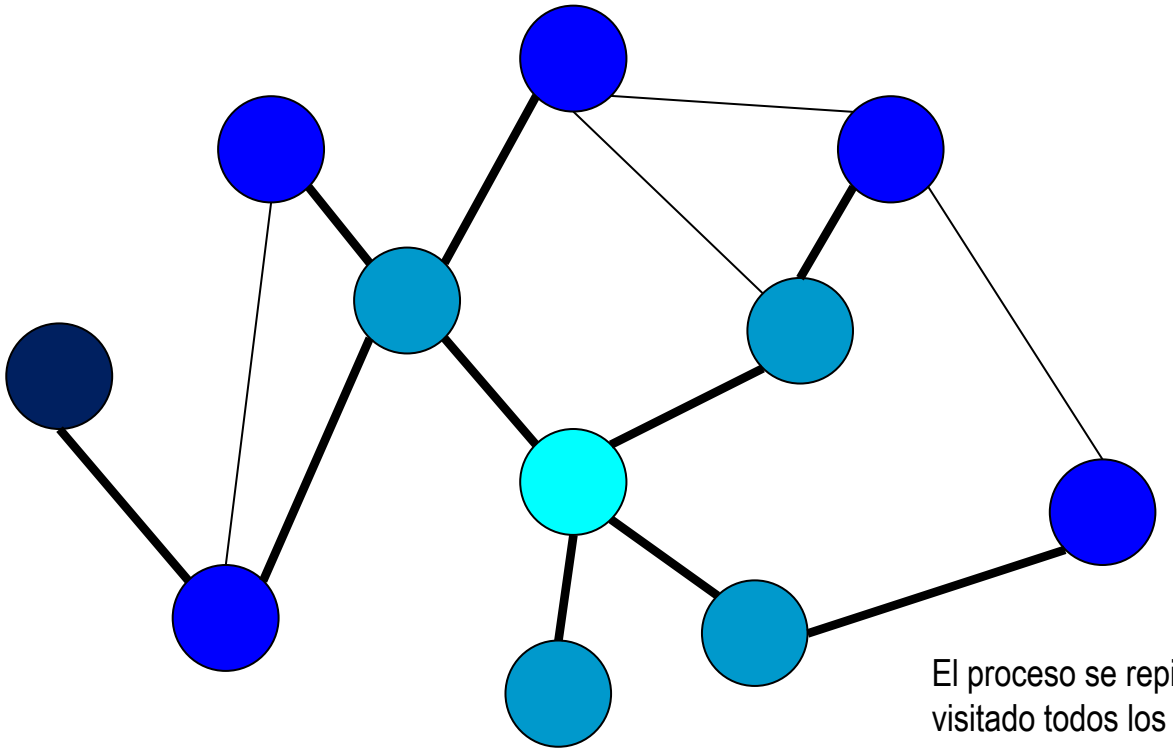
Se visitan otros vecinos de su vecindario 1…

Recorrido en anchura



…hasta terminar con el vecindario nivel 2

Recorrido en anchura



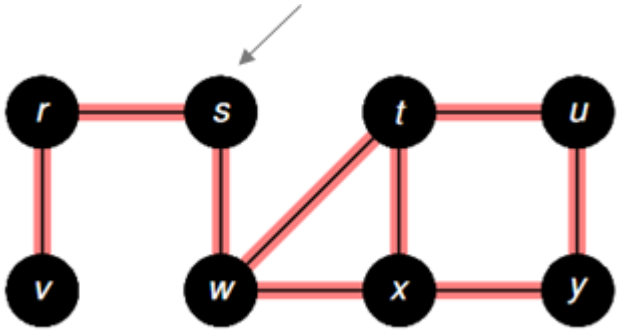
El proceso se repite hasta haber visitado todos los nodos

Recorrido en anchura: algoritmo

```
algoritmo recorrido_en_anchura (grafo, nodo_inicial)
inicio
  inicializar(cola)
  marcar(nodo_inicial)
  enqueue(cola, nodo_inicial)
  mientras not es_vacia (cola) hacer
    nodo_actual := first(cola)
    dequeue(cola)
    procesar(nodo_actual)
    para todo nodo_aux sin marcar y adyacente a nodo_actual hacer
      marcar(nodo_aux)
      enqueue(cola, nodo_aux)
    fin_para
  fin_mientras
fin recorrido_en_anchura
```

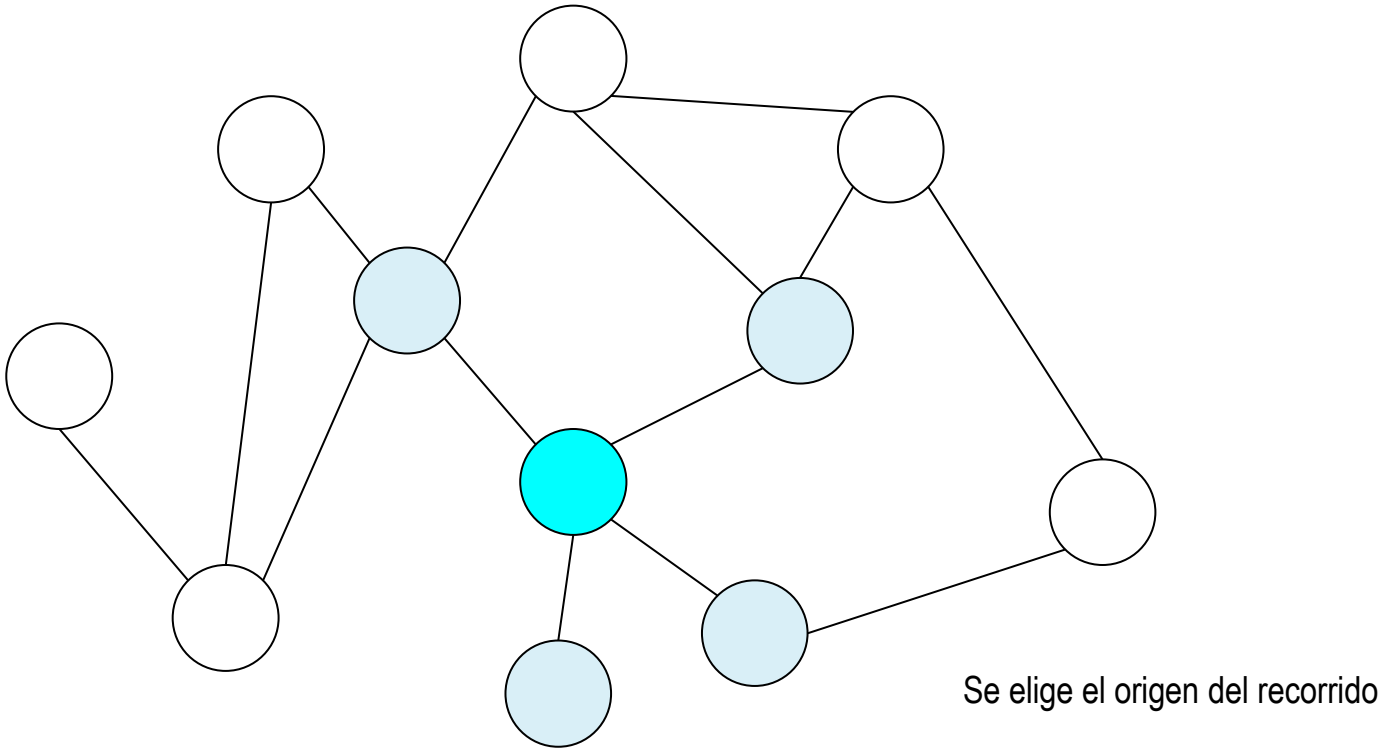
Recorrido en anchura: algoritmo

```
algoritmo recorrido_en_anchura (grafo, nodo_inicial)
inicio
  inicializar(cola)
  marcar(nodo_inicial)
  enqueue(cola, nodo_inicial)
  mientras not es_vacia (cola) hacer
    nodo_actual := first(cola)
    dequeue(cola)
    procesar(nodo_actual)
    para todo nodo_aux sin marcar y adyacente a nodo_actual hacer
      marcar(nodo_aux)
      enqueue(cola, nodo_aux)
    fin_para
  fin_mientras
fin recorrido_en_anchura
```

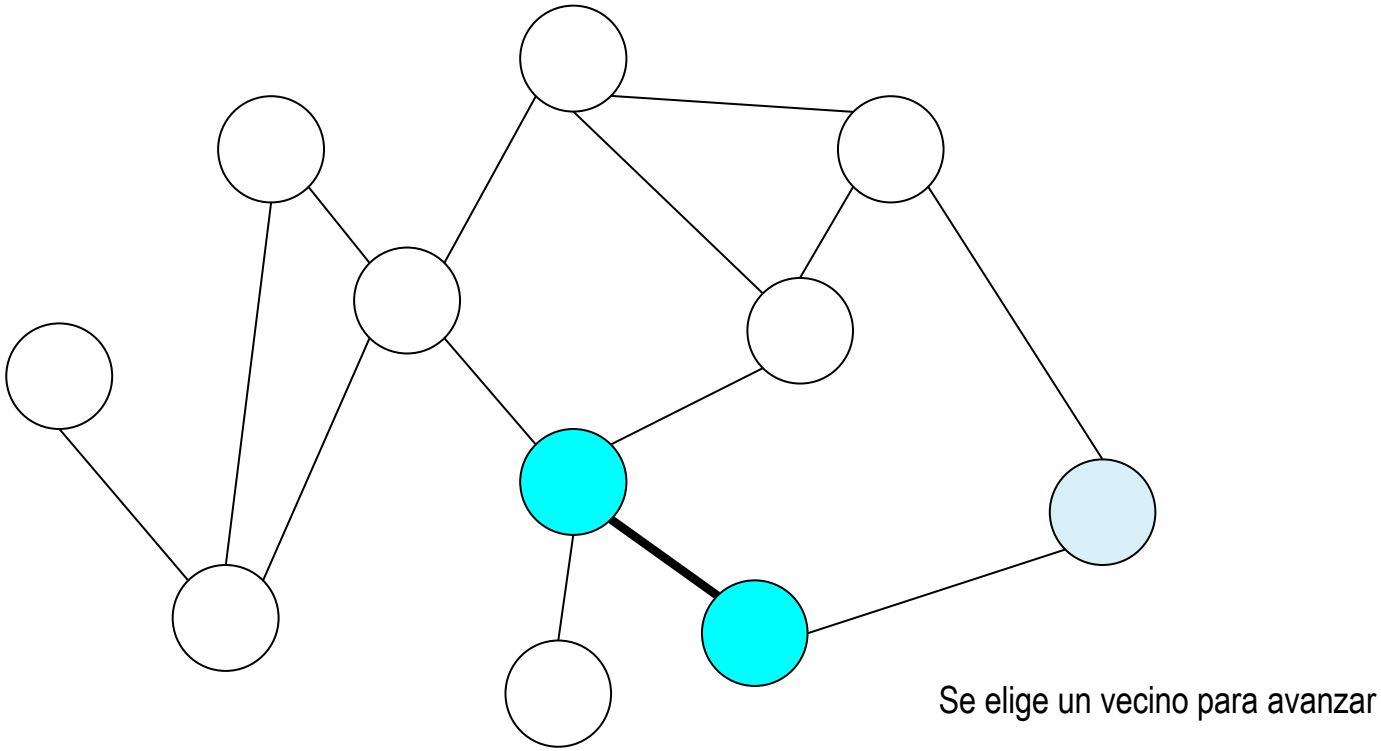


S // [ ] // WR →
R // RTX →
TX // TXV →
XV // XVU →
VU // VUY →
UY →
Y →
[ ]

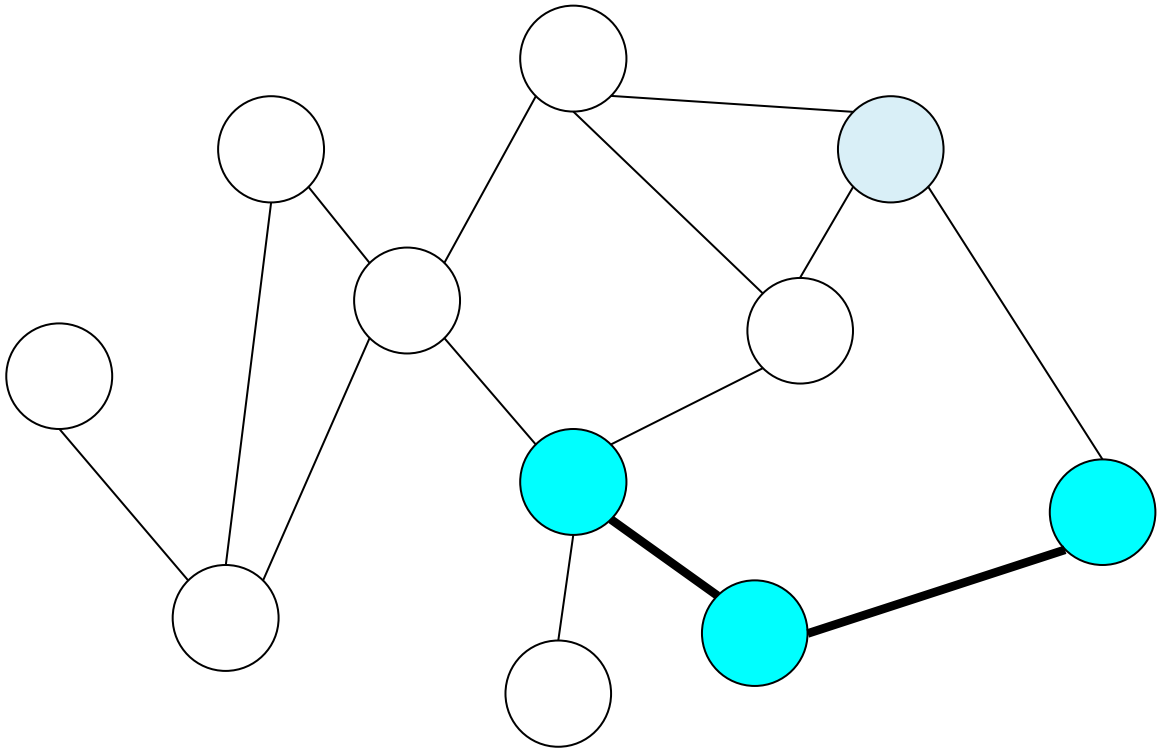
Recorrido en profundidad (DFS)



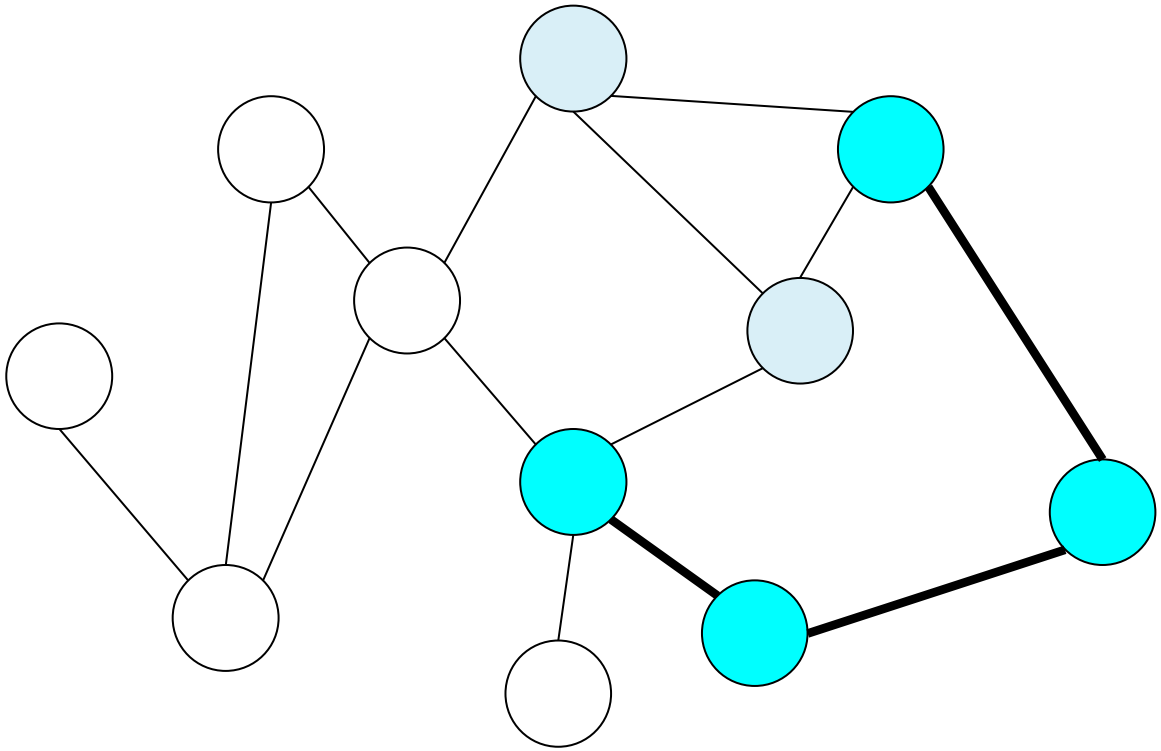
Recorrido en profundidad (DFS)



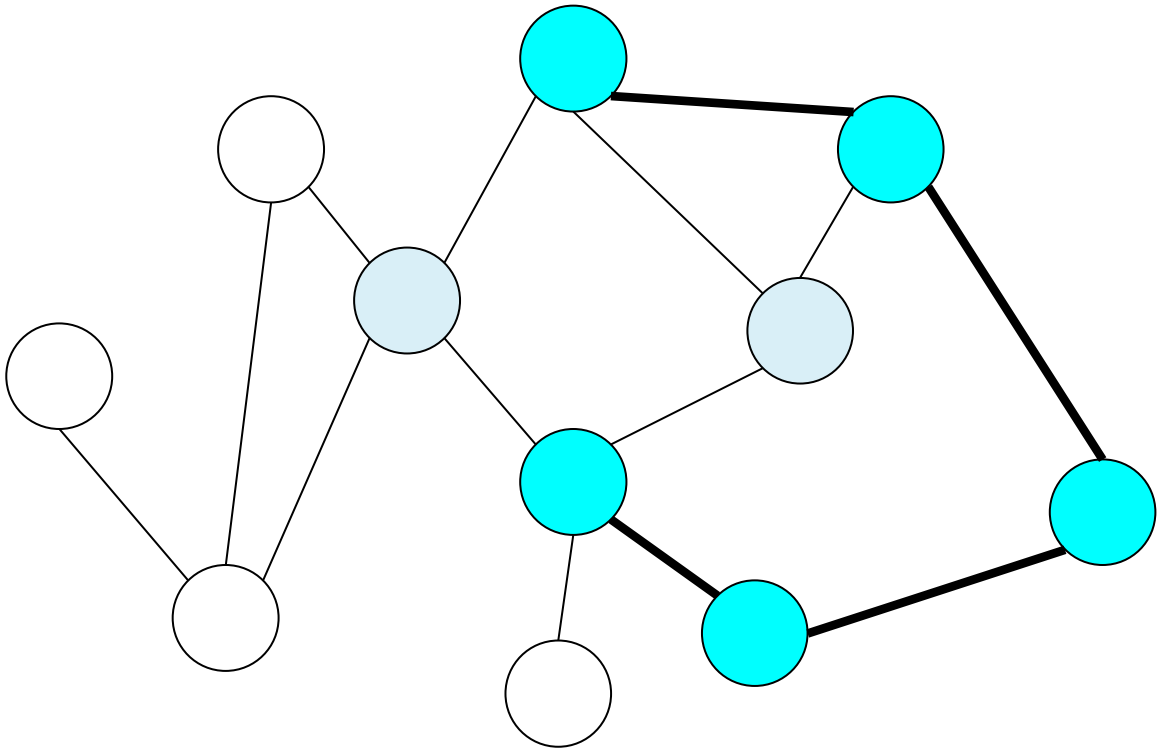
Recorrido en profundidad (DFS)



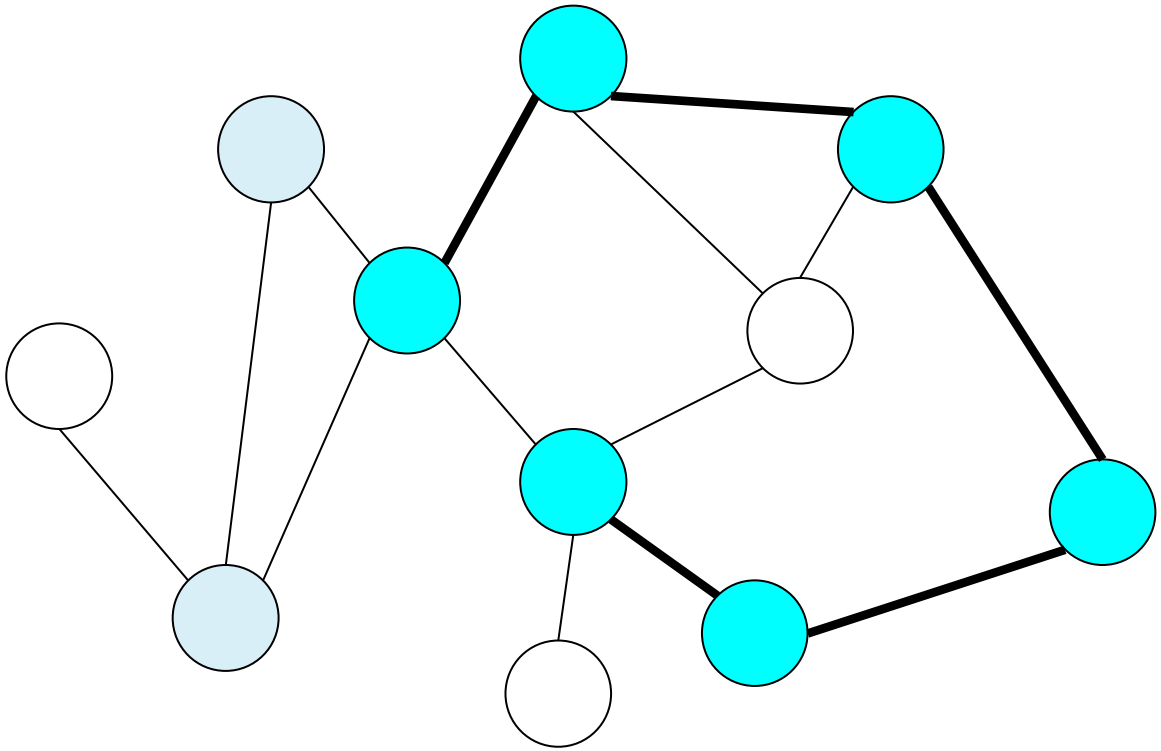
Recorrido en profundidad (DFS)



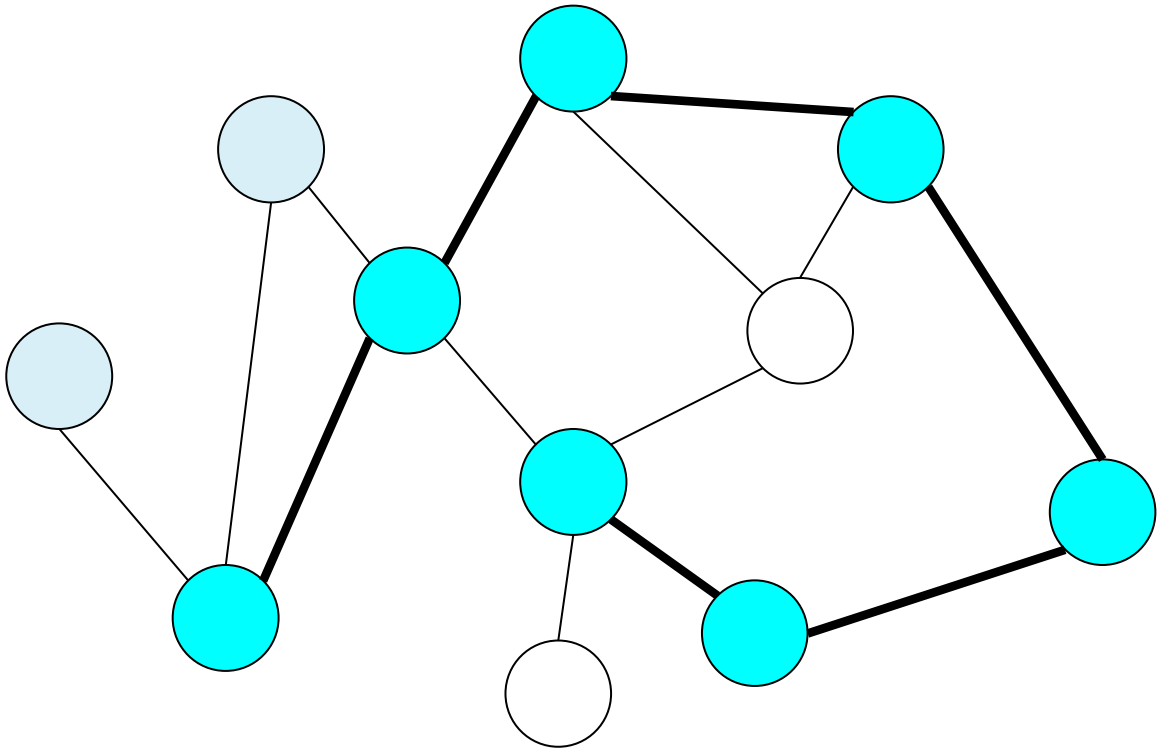
Recorrido en profundidad (DFS)



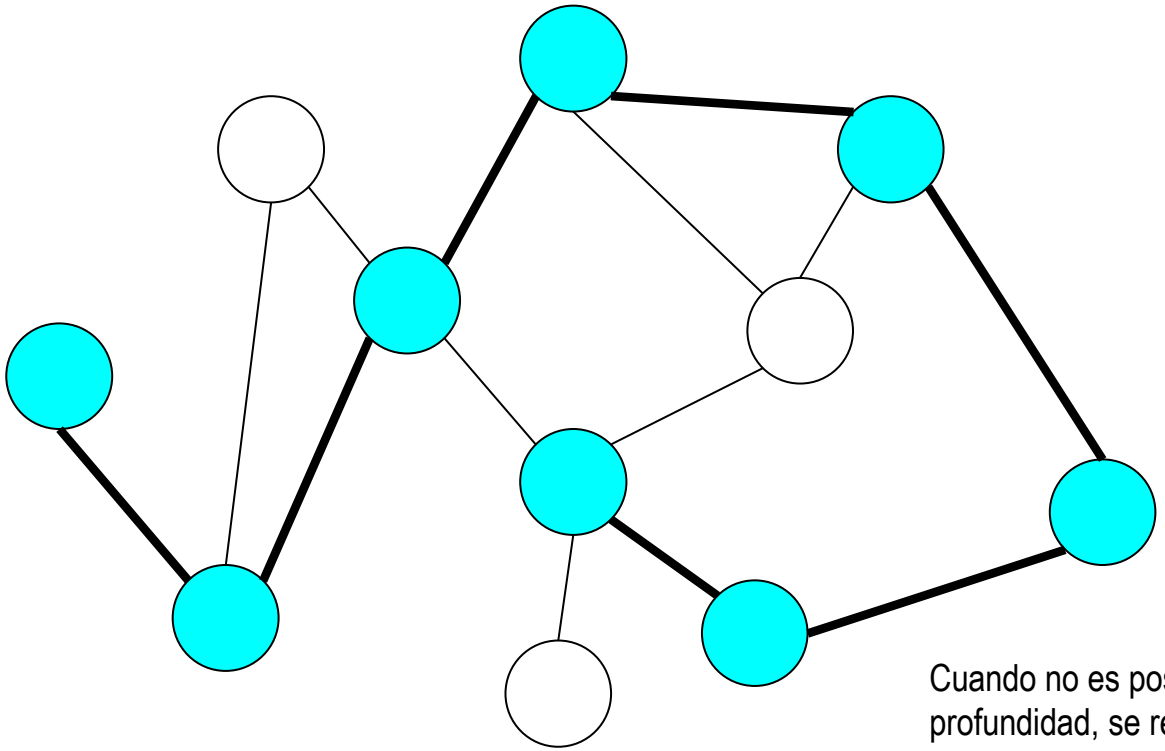
Recorrido en profundidad (DFS)



Recorrido en profundidad (DFS)

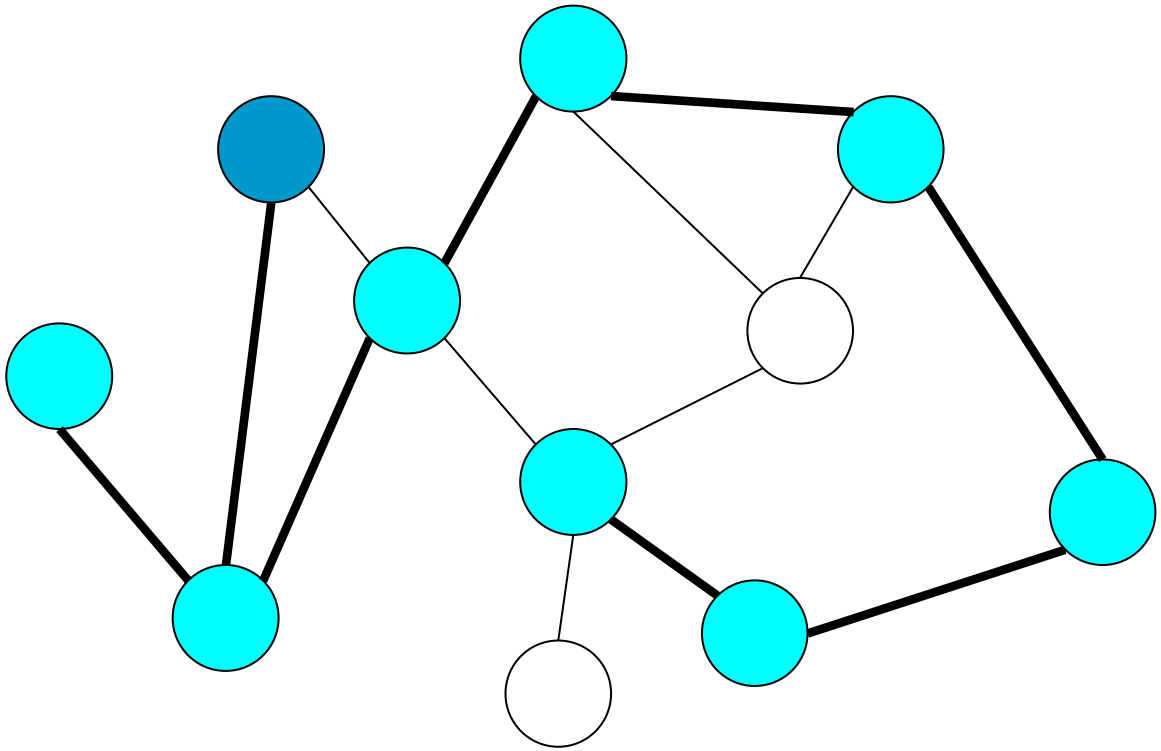


Recorrido en profundidad (DFS)

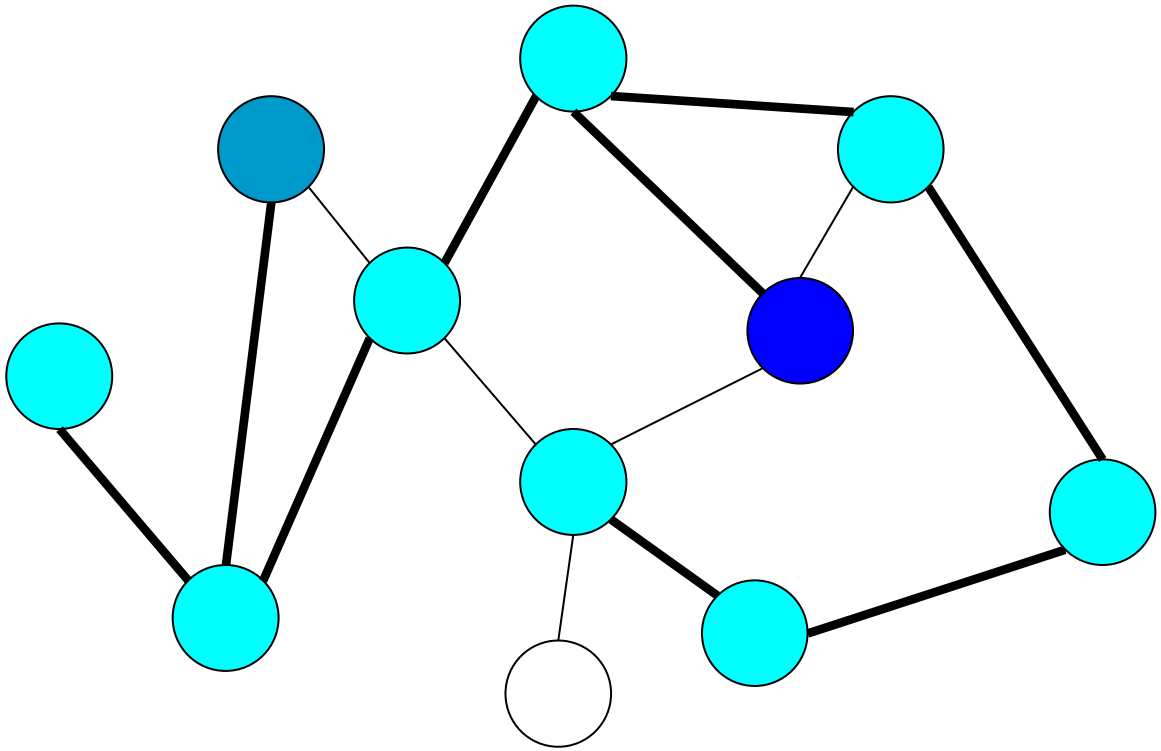


Cuando no es posible seguir avanzando en profundidad, se retoman caminos no tomados

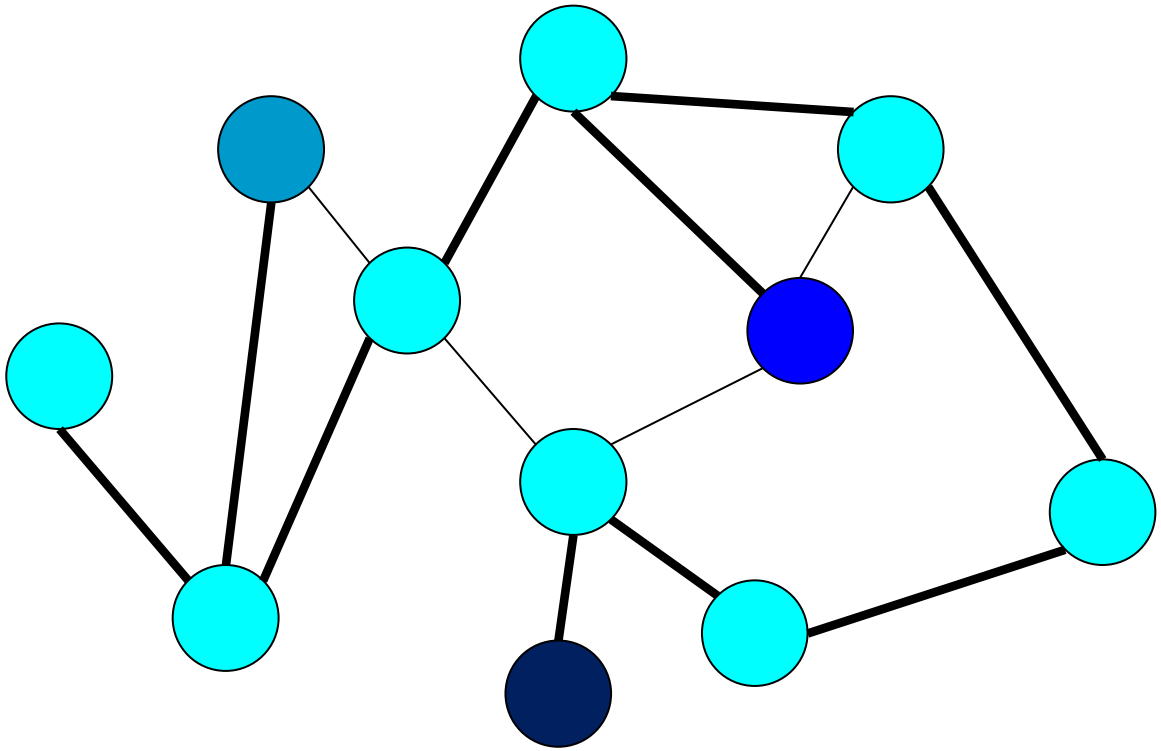
Recorrido en profundidad (DFS)



Recorrido en profundidad (DFS)



Recorrido en profundidad (DFS)

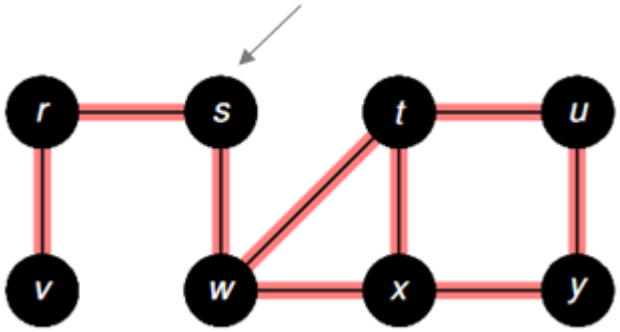


Recorrido en profundidad: algoritmo

```
algoritmo recorrido_en_profundidad (grafo, nodo_inicial)
inicio
  inicializar(pila)
  marcar(nodo_inicial)
  push(pila, nodo_inicial)
  mientras not es_vacia (pila) hacer
    nodo_actual := peek(pila)
    pop(pila)
    procesar(nodo_actual)
    para todo nodo_aux sin marcar y adyacente a nodo_actual hacer
      marcar(nodo_aux)
      push(pila, nodo_aux)
    fin_para
  fin_mientras
fin recorrido_en_profundidad
```

Recorrido en profundidad: algoritmo

```
algoritmo recorrido_en_profundidad (grafo, nodo_inicial)
inicio
  inicializar(pila)
  marcar(nodo_inicial)
  push(pila, nodo_inicial)
  mientras not es_vacia (pila) hacer
    nodo_actual := peek(pila)
    pop(pila)
    procesar(nodo_actual)
    para todo nodo_aux sin marcar y adyacente a nodo_actual hacer
      marcar(nodo_aux)
      push(pila, nodo_aux)
    fin_para
  fin_mientras
fin recorrido_en_profundidad
```

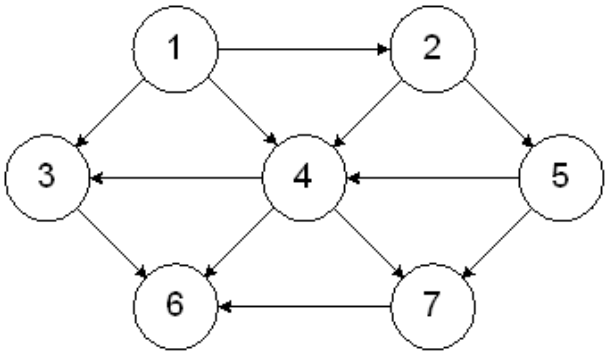


S // [ ] // RW →
W // VW →
W →
[ ] // XT →
T // YT →
T // UT →
T →
[ ]

Ejercicios propuestos

Ejercicio 1

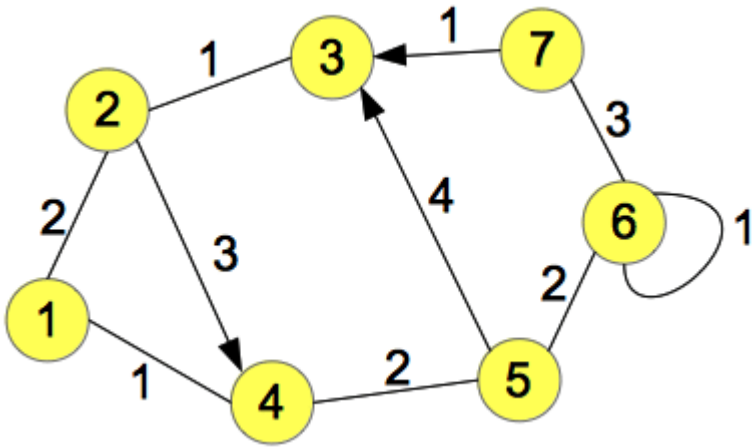
- Añade al TAD Grafo binario una función que permita calcular el grado saliente de un nodo (*out-degree*), es decir, el número de relaciones con otros nodos del grafo.
- Prográmalo para ambas implementaciones: matriz y lista de adyacencia



En este ejemplo el out-degree del nodo 5 es 2, mientras que el del 4 es 3

Ejercicio 2

- Completa el TAD Grafo valorado con otra función que compute el peso de cada nodo en función de los pesos de sus enlaces salientes y devuelva dichos valores en una lista donde habrá un peso para cada nodo.
- Prográmalo para ambas implementaciones: matriz y lista de adyacencia

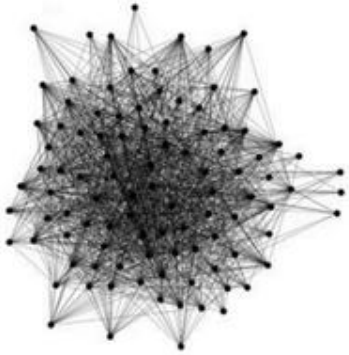


En este ejemplo se retornaría la lista siguiente: [3,6,1,3,4,6,4] donde 3 es el peso del out-degree del nodo 1, 6 el del nodo 2 y así sucesivamente.

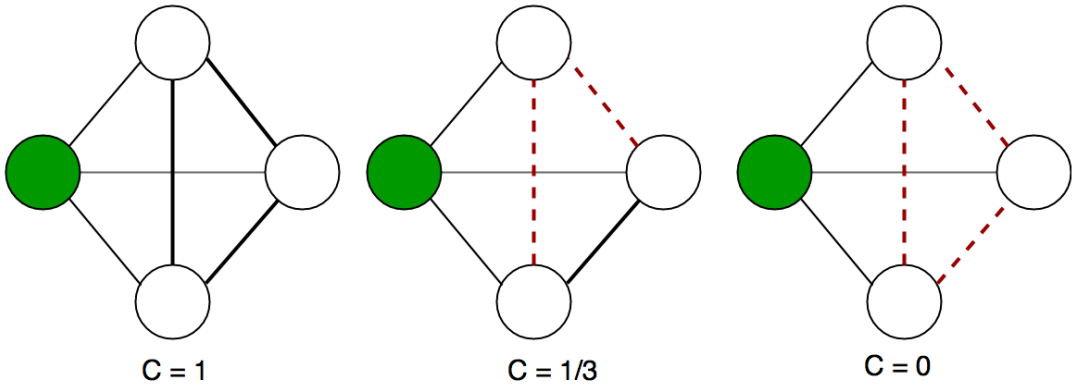
Ejercicio 3

- Añade al TAD Grafo binario una función que permita calcular su densidad, sabiendo que la densidad de un grafo binario es un valor real entre 0 y 1 que representa el número de arcos que tiene el grafo dividido entre el número potencial de arcos que podría llegar a tener.
- Prográmalo para ambas implementaciones: matriz y lista de adyacencia

Grafo disperso



Grafo denso

Ejercicio 4

- Añade al TAD Grafo binario una función que permita calcular el coeficiente de agrupamiento (o *clustering)* de un nodo. Dicho coeficiente se calcula como la densidad del grafo que forman los nodos de su vecindario.
- Prográmalo para ambas implementaciones: matriz y lista de adyacencia